PROJECT TITLE:

MARKETING SPEND VS CUSTOMER WALK-INS

HARDWARE & PLYWOOD RETAIL INTELLIGENCE CENTER

Comprehensive Project Report

Project Domain: Full Stack Web Application with Analytics and AI Forecasting

Technology Stack: React, TypeScript, Vite, Express, MySQL, Python LightGBM

ABSTRACT

This report presents the design, development, and validation of a production-oriented web application for a single-store hardware and plywood business. The system integrates inventory management, stock updates, billing operations, sold-details tracking, marketing analytics, inventory analytics, and machine-learning-based forecasting into one unified digital platform. The frontend is developed using React, TypeScript, and Vite, while the backend uses Express and MySQL for transactional persistence and structured reporting. The platform supports operational workflows such as product creation, stock replenishment, billing with optional customer email and downloadable PDF bill generation, as well as executive dashboards and analytics narratives to improve business decision quality.

In addition to standard CRUD and reporting capabilities, the system introduces an AI Forecast Lab that invokes a Python LightGBM model through backend orchestration to estimate future sales units, walk-ins, and recommended marketing spend under configurable risk profiles. The architecture follows API-first principles so that both web and mobile clients can consume the same authoritative backend data. The implemented system addresses practical constraints of small retail businesses by delivering low-friction usability, transparent data flows, and near real-time updates.

The report documents complete lifecycle phases: requirements, analysis, architecture, module design, schema design, implementation details, testing approach, performance considerations, and future enhancement paths. The resulting application demonstrates that a modern full stack architecture can significantly improve inventory visibility, billing accuracy, and marketing-performance interpretation in a real retail setting.

TABLE OF CONTENTS

- 1. INTRODUCTION
- 2. LITERATURE SURVEY
- 3. SYSTEM ANALYSIS
- 4. SYSTEM SPECIFICATION
- 5. SYSTEM DESIGN
- 6. MODULE DESCRIPTION
- 7. IMPLEMENTATION AND TESTING
- 8. PERFORMANCE, DEPLOYMENT, AND OPERATIONS
- 9. CONCLUSION AND FUTURE ENHANCEMENTS
- 10. APPENDICES
- 11. REFERENCES

LIST OF ABBREVIATIONS AND ACRONYMS

Abbreviation	Meaning
UI	User Interface
UX	User Experience
API	Application Programming Interface
RDBMS	Relational Database Management System
CRUD	Create, Read, Update, Delete
KPI	Key Performance Indicator
ML	Machine Learning
CNN	Convolutional Neural Network
ROI	Return on Investment
CSV	Comma-Separated Values
SMTP	Simple Mail Transfer Protocol

CHAPTER 1

1. INTRODUCTION

1.1 OBJECTIVE OF THE PROJECT

The objective of this project is to build an integrated digital operations platform for a hardware and plywood store that can replace fragmented spreadsheet-based workflows with a reliable, auditable, and scalable web system. The application aims to manage inventory records, support daily stock updates, generate billing transactions, maintain sold-product history, and convert raw operational data into actionable intelligence through analytics and forecasting.

1.2 SCOPE OF THE PROJECT

The scope includes frontend user experience, backend API design, MySQL schema design, business validation rules, PDF bill generation, email notification support, data import/export, and analytics visualizations. The project is designed for single-store retail operations but follows modular architecture patterns that enable extension to multiple branches with minimal structural changes.

1.3 OUTLINE OF THE PROJECT

The report is structured from requirement framing to implementation and deployment. Initial chapters explain problem context and existing system limitations; middle chapters define architecture, schema, and module workflows; later chapters document testing, optimization, and operational runbook practices.

1.4 DOMAIN EXPLANATION

Retail inventory intelligence requires synchronization of stock quantities, valuation metrics, movement logs, and customer billing transactions. This domain is highly event-driven: stock additions, sales deductions, and settings changes must remain consistent across all modules. The project handles this by centralizing persistence in MySQL and exposing deterministic REST APIs consumed by the UI.

CHAPTER 2

2. LITERATURE SURVEY

Modern retail software increasingly converges on web-first architectures using component-based frontend frameworks and service-oriented backends. Industry implementations show that maintaining a strict separation between presentation, business logic, and persistence significantly improves maintainability and extensibility. React ecosystems are preferred for responsive admin interfaces, while Express-based APIs remain common for rapid delivery and integration flexibility.

Data-driven retail systems frequently use monthly and product-level aggregation models for KPI dashboards, stock health, and marketing-attribution analysis. A common practical challenge is data quality and temporal consistency. This project adopts canonical month keys and derived metrics to keep analytical interpretations stable over time.

Recent adoption of machine learning in retail decision support has shifted from purely descriptive dashboards to predictive recommendation workflows. Lightweight gradient boosting approaches such as LightGBM provide strong performance for tabular business data and are suitable for server-invoked forecasting in mixed JavaScript/Python stacks.

CHAPTER 3

3. SYSTEM ANALYSIS

3.1 PROBLEM DEFINITION

The pre-digital process relied on disconnected records for stock, product pricing, and sales movement. This led to slow reconciliation, inconsistent quantities, delayed insights, and limited confidence in decision making.

3.2 EXISTING SYSTEM

In the existing environment, manual records and ad-hoc spreadsheets handled inventory and billing. While simple to start, the approach lacked transactional integrity, audit trails, and consolidated reporting.

3.3 DISADVANTAGES OF EXISTING SYSTEM

- No unified source of truth across inventory, billing, and stock updates.
- Higher risk of quantity mismatches and valuation errors.
- No integrated PDF billing and limited customer communication tracking.
- Delayed visibility into low-stock alerts and idle inventory.
- No predictive planning support for marketing spend decisions.

3.4 PROPOSED SYSTEM

The proposed system delivers a centralized full stack web platform with API-driven data persistence and analytics-aware module boundaries. Operational updates immediately flow into dashboards and downstream analytical views.

3.5 ADVANTAGES OF PROPOSED SYSTEM

- Single data backbone using MySQL for consistency and durability.
- Fast billing-to-stock synchronization with generated bill numbers and optional email.
- Role-friendly dashboards with KPI, trends, and narrative insights.
- Configurable business settings including categories, units, currency, and theme.
- Integrated AI forecast endpoint for decision support.

CHAPTER 4

4. SYSTEM SPECIFICATION

4.1 HARDWARE REQUIREMENTS

Component	Minimum Requirement
Processor	Dual-core 2.0 GHz or above
RAM	4 GB (8 GB recommended)
Storage	5 GB free (including Node modules and database)
Display	1366x768 or above
Network	Localhost or LAN for multi-device testing

4.2 SOFTWARE REQUIREMENTS

Software	Version / Notes
Operating System	Windows 10/11
Node.js	LTS release
npm	Bundled with Node.js
MySQL Server	8.x preferred
VS Code	Latest stable
Python	3.x for ML model execution

4.3 SOFTWARE STACK AND DEPENDENCIES

Key dependency groups are listed below based on the active package manifest.

Layer	Major Libraries
Frontend	React, React Router, Zustand, Recharts, Framer Motion, Zod, React Hook Form

Backend	Express, mysql2, nodemailer, pdfkit, papaparse
Build/Dev	Vite, TypeScript, concurrently, Tailwind CSS
ML Integration	Python script with LightGBM-compatible pipeline

Project scripts: build, dev, dev:client, dev:server, normalize:data, preview, start:server

CHAPTER 5

5. SYSTEM DESIGN

5.1 PROPOSED ARCHITECTURE

The architecture follows a layered model. The React frontend consumes REST APIs from the Express backend. The backend enforces validation and business logic before persisting data in MySQL. AI forecasting requests are delegated from Node.js to a Python process and the result is returned through the same API layer.

5.2 DATABASE DESIGN

The core schema consists of operational and configuration tables summarized below.

Table	Approx. Columns	Purpose
inventory_records	24	Stores month-wise product inventory metrics and valuation fields.
stock_logs	12	Tracks stock addition history and operational notes.
product_images	3	Stores base64 product image payloads by product name.
app_settings	8	Stores global business settings for UI and reporting behavior.
billing_sales	16	Stores generated billing transactions and email delivery status.

5.3 API DESIGN

Implemented REST endpoints discovered from backend source are listed below.

Method	Endpoint	Operational Role

GET	/api/health	Service health check for readiness monitoring
GET	/api/bootstrap	Single-shot payload hydration for frontend state
POST	/api/ml/forecast	Runs forecasting pipeline and returns recommendation
POST	/api/records/import	Bulk imports inventory dataset
POST	/api/products	Creates products / inventory records
PATCH	/api/products/:productName	Updates existing product/inventory values
POST	/api/stock-logs	Creates stock movement entries
POST	/api/billing/sales	Generates bill, stock deduction, and email workflow
GET	/api/billing/sales/:id/pdf	Renders bill PDF for preview/download
DELETE	/api/stock-logs/:id	Deletes stock log entry
DELETE	/api/product-images/:productName	Stores or removes product image asset
PATCH	/api/settings	Reads/updates global application settings
DELETE	/api/data	Clears operational data store

5.4 FRONTEND ROUTE DESIGN

The web application is organized as independent route modules. Active routes detected from source are:

- Dashboard
- InventoryAnalytics
- InventoryLog
- Login
- MLForecastLab
- MarketingAnalytics
- Products
- SalesMovement
- Settings
- SoldDetails

5.5 ANALYTICS DESIGN PRINCIPLES

Analytics computation uses deterministic helper functions for month-key mapping, stock status classification, category totals, top products, low-stock detection, and health scoring. This approach keeps visualizations and narrative summaries reproducible and reduces divergence between UI components.

CHAPTER 6

6. MODULE DESCRIPTION

6.1 Dashboard MODULE

Executive overview module presenting health score, KPI cards, top movers, and business narrative.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.2 InventoryAnalytics MODULE

Stock health and idle inventory analysis module with alert-friendly visualization.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.3 InventoryLog MODULE

Stock update module focused on incoming stock recording and stock update history tracking.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.4 Login MODULE

Authentication gateway module with credential validation and protected route entry.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.5 MLForecastLab MODULE

Decision support module that invokes backend ML forecasting and recommendation pipeline.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.6 MarketingAnalytics MODULE

Trend analysis module for spend vs walk-ins, conversions, purchases, and ROI insighting.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.7 Products MODULE

Catalog management module for filtering products, editing details, and image upload/preview workflows.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.8 SalesMovement MODULE

Billing module for customer checkout, bill generation, PDF bill preview, and stock deduction.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.9 Settings MODULE

Administrative module for store configuration, import/export behavior, and local preference controls.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

6.10 SoldDetails MODULE

Transaction history module for sold-item records and email status verification.

Inputs, validations, and side effects are coordinated through centralized store actions to guarantee that the module remains synchronized with backend state. Each module is designed to remain independently maintainable while still participating in a shared business data lifecycle.

From an implementation perspective, this module follows consistent UI patterns: typed forms, schema-aware data transformations, and API-backed persistence. This consistency reduces defects and simplifies onboarding for future contributors who need to extend or refactor project features.

CHAPTER 7

7. IMPLEMENTATION AND TESTING

7.1 IMPLEMENTATION WORKFLOW

Implementation followed an incremental module-by-module process: schema setup, API readiness, frontend route delivery, state integration, analytics validation, and final stabilization. Build and runbook practices were documented to ensure repeatable startup after system shutdown.

7.2 TESTING STRATEGY

Testing includes unit-level verification of helper logic, integration-level checks across store actions and API calls, and validation-level checks for business behavior such as stock reconciliation after billing. Runtime smoke testing and build verification are part of release-readiness criteria.

7.3 SAMPLE TEST CASES

TC ID	Scenario	Expected Result	Status
TC-01	Load bootstrap payload	Dashboard renders without error and data appears	Pass
TC-02	Add product	Product persists and appears in catalog	Pass
TC-03	Upload product image	Image maps to product and preview works	Pass
TC-04	Add stock log	Stock history row created and values visible	Pass
TC-05	Generate billing sale	Bill stored and stock decremented	Pass

TC-06	Preview bill PDF	PDF opens with valid transaction details	Pass
TC-07	Email optional flow	Billing completes when customer email is empty	Pass
TC-08	Analytics charts render	Monthly trend and inventory status visualizations show	Pass
TC-09	ML forecast invocation	Forecast response returns recommendation payload	Pass
TC-10	Build pipeline	TypeScript and Vite build succeed	Pass

7.4 VALIDATION COMMANDS

- npm install
- npm run dev
- npm run build
- npm run preview
- GET /api/health

CHAPTER 8

8. PERFORMANCE, DEPLOYMENT, AND OPERATIONS

8.1 PERFORMANCE CONSIDERATIONS

Performance work included route-level lazy loading, state hydration management, and product-image rendering optimization. Since image-heavy payloads can freeze browser rendering, practical mitigations include lazy decoding, pagination, and controlled upload compression for future assets.

8.2 LOCAL NETWORK HOSTING

For LAN access testing, frontend can be served with host binding while backend remains on API port. This allows mobile device testing on the same Wi-Fi network without external hosting infrastructure.

- Backend: npm run dev:server
- Frontend LAN mode: npm run dev:client -- --host 0.0.0.0 --port 5173
- Network URL format: http://<local-ip>:5173

8.3 STARTUP AND RUNBOOK ALIGNMENT

Operational startup must begin with MySQL service readiness, followed by backend and frontend server startup. The documented runbook in this project ensures reproducible launches after complete system shutdown and helps reduce time-to-recovery during demonstrations or evaluations.

CHAPTER 9

9. CONCLUSION AND FUTURE ENHANCEMENTS

9.1 CONCLUSION

The completed system demonstrates a coherent, production-minded web platform for hardware and plywood retail management. It unifies inventory tracking, billing operations, analytics interpretation, and forecast-assisted planning under a single interface. The architecture is maintainable, extensible, and compatible with cross-client consumption patterns.

9.2 FUTURE ENHANCEMENTS

- Role-based authorization with multi-user access controls.
- Branch-aware multi-store inventory partitioning.
- Advanced image storage migration from base64 to object storage/CDN.
- Automated regression tests and CI pipelines.
- Forecast model monitoring and drift alerts.
- Offline-first synchronization strategy for constrained networks.

CHAPTER 10

10. APPENDICES

10.1 SOURCE CODE EXCERPTS

Appendix A - Store Layer (src/lib/store.ts)

```
// Workflow: Zustand store orchestrating app state, backend sync actions, and UI settings
state.
import { create } from "zustand";
import { AppSettings, BillingSaleEntry, InventoryRecord, StockLogEntry } from "./types";
import { apiRequest } from "./api";

// Defaults are used both for first-run UI and for fallback state when API boot fails.
const defaultSettings: AppSettings = {
  storeName: "Oakline Hardware & Plywood",
  currency: "INR",
  categories: ["Glass", "Plywood", "Hardwares", "Beeding", "Laminates", "Doors"],
  units: ["sq ft", "sheet", "pcs", "box", "pack", "set", "ft", "kg"],
  pricePresets: [65, 120, 240, 850, 1500, 3200],
  theme: "dark",
};

// Central store contract used by route screens and shared components.
type AppState = {
  records: InventoryRecord[];
  stockLogs: StockLogEntry[];
  billingSales: BillingSaleEntry[];
  productImages: Record<string, string>;
  settings: AppSettings;
  boot: () => Promise<void>;
  importRecords: (records: InventoryRecord[]) => Promise<void>;
  updateProduct: (productName: string, updates: Partial<InventoryRecord>) => Promise<void>;
  addProduct: (record: InventoryRecord) => Promise<void>;
  addStockLog: (entry: Omit<StockLogEntry, "id">) => Promise<void>;
  createBillingSale: (payload: {
    customer_name: string;
    customer_email?: string;
    category: string;
    product_name: string;
```

```
    units_sold: number;
  }) => Promise<{ id: string; bill_number: string; email_status: string; email_error?: string
}>;
  removeStockLog: (id: string) => Promise<void>;
  setProductImage: (productName: string, image: string) => Promise<void>;
  removeProductImage: (productName: string) => Promise<void>;
  setTheme: (theme: AppSettings["theme"]) => Promise<void>;
  updateSettings: (updates: Partial<AppSettings>) => Promise<void>;
  clearData: () => Promise<void>;
};

type BootstrapPayload = {
  records: InventoryRecord[];
  stockLogs: StockLogEntry[];
  billingSales: BillingSaleEntry[];
  productImages: Record<string, string>;
  settings: AppSettings;
};

let bootPromise: Promise<void> | null = null;
let queuedBoot = false;

// Theme is mirrored onto the document body to allow CSS theme selectors.
const applyTheme = (theme: AppSettings["theme"]) => {
  if (typeof document !== "undefined") {
    document.body.dataset.theme = theme;
  }
};

const normalizeSettings = (settings?: Partial<AppSettings>): AppSettings => ({
  ...defaultSettings,
  ...settings,
  // Keep default presets when empty arrays are provided from partially configured payloads.
  pricePresets:
    settings?.pricePresets && settings.pricePresets.length
      ? settings.pricePresets
      : defaultSettings.pricePresets,
});

// Store actions mostly follow the pattern: call API -> refresh snapshot via boot().
export const useAppStore = create<AppState>((set, get) => ({
```

```
records: [],
stockLogs: [],
billingSales: [],
productImages: {},
settings: defaultSettings,
// boot() is the canonical hydration path used after app start and after server mutations.
boot: async () => {
  if (bootPromise) {
    return bootPromise;
  }

  bootPromise = (async () => {
    try {
      const payload = await apiRequest<BootstrapPayload>("/bootstrap");
      const settings = normalizeSettings(payload.settings);
      set({
        records: payload.records,
        stockLogs: payload.stockLogs,
        billingSales: payload.billingSales,
        productImages: payload.productImages,
        settings,
      });
      applyTheme(settings.theme);
    } catch (error) {
      console.error("Failed to load bootstrap data from API", error);
      const settings = normalizeSettings(defaultSettings);
      set({ records: [], stockLogs: [], billingSales: [], productImages: {}, settings });
      applyTheme(settings.theme);
    }
  })().finally(() => {
    bootPromise = null;
    if (queuedBoot) {
      queuedBoot = false;
      void get().boot();
    }
  });

  return bootPromise;
},
```

Appendix B - API Layer (server/src/index.js)

```
// Workflow: Express API server handling data persistence, billing operations, email delivery,
and PDF generation.
import cors from "cors";
import dotenv from "dotenv";
import express from "express";
import { spawn } from "node:child_process";
import fs from "node:fs/promises";
import dns from "node:dns/promises";
import path from "node:path";
import { fileURLToPath } from "node:url";
import mysql from "mysql2/promise";
import nodemailer from "nodemailer";
import Papa from "papaparse";
import PDFDocument from "pdfkit";

dotenv.config();

const app = express();
const PORT = Number(process.env.API_PORT ?? 4000);
const FRONTEND_ORIGIN = process.env.FRONTEND_ORIGIN ?? "http://localhost:5173";
const currentFile = fileURLToPath(import.meta.url);
const currentDir = path.dirname(currentFile);
const ML_SCRIPT_PATH = path.resolve(currentDir, "../ml/train_forecast.py");
const ML_PYTHON_BIN = String(process.env.ML_PYTHON_BIN ?? "").trim();
const ML_SCRIPT_TIMEOUT_MS = Number(process.env.ML_SCRIPT_TIMEOUT_MS ?? 45000);
const GMAIL_SMTP_USER = String(process.env.GMAIL_SMTP_USER ?? "").trim();
const GMAIL_SMTP_APP_PASSWORD = String(process.env.GMAIL_SMTP_APP_PASSWORD ??
"").replace(/\s+/g, "").trim();
const GMAIL_SMTP_FROM = String(process.env.GMAIL_SMTP_FROM ?? GMAIL_SMTP_USER).trim();
const GMAIL_SMTP_TIMEOUT_MS = Number(process.env.GMAIL_SMTP_TIMEOUT_MS ?? 15000);

const DB_CONFIG = {
  host: process.env.DB_HOST ?? "localhost",
  port: Number(process.env.DB_PORT ?? 3306),
  user: process.env.DB_USER ?? "root",
  password: process.env.DB_PASSWORD ?? "",
  database: process.env.DB_NAME ?? "hardware_store_db",
};
```

```
const VALID_STATUSES = new Set(["Healthy", "Low Stock", "Reorder Soon", "Out of Stock"]);

const DEFAULT_SETTINGS = {
  storeName: "Hardware & Plywood Store",
  currency: "INR",
  categories: ["Glass", "Plywood", "Hardwares", "Beeding", "Laminates", "Doors"],
  units: ["sq ft", "sheet", "pcs", "box", "pack", "set", "ft", "kg"],
  pricePresets: [65, 120, 240, 850, 1500, 3200],
  theme: "dark",
};

const pad2 = (value) => String(value).padStart(2, "0");

let pool;

app.use(cors({ origin: FRONTEND_ORIGIN }));
app.use(express.json({ limit: "25mb" }));

// Primitive converters and time/currency formatters used throughout request handlers.
const toNumber = (value, fallback = 0) => {
  const numeric = Number(value);
  return Number.isFinite(numeric) ? numeric : fallback;
};

const toLocalDateTime = (date = new Date()) => {
  const year = date.getFullYear();
  const month = pad2(date.getMonth() + 1);
  const day = pad2(date.getDate());
  const hours = pad2(date.getHours());
  const minutes = pad2(date.getMinutes());
  const seconds = pad2(date.getSeconds());
  return {
    saleDate: `${year}-${month}-${day}`,
    saleTime: `${hours}:${minutes}:${seconds}`,
  };
};

const formatCurrencyShort = (amount, currency) => {
  const value = toNumber(amount, 0);
  const normalizedCurrency = String(currency || "INR").toUpperCase();
```

```
  if (normalizedCurrency === "INR") {
    return `Rs ${Math.round(value)}`;
  }
  return `${normalizedCurrency} ${Math.round(value)}`;
};

const normalizeEmailAddress = (email) => String(email ?? "").trim().toLowerCase();

const isValidEmailAddress = (email) => /^[^\s@]+@[^\s@]+\.[^\s@]+$/.test(String(email ??
"").trim());

const extractEmailAddress = (value) => {
  const text = String(value ?? "").trim();
  const match = text.match(/<([^<>]+)>/);
  return match ? match[1] : text;
};

// Recipient-domain validation: prefer MX records, then fall back to A/AAAA for edge
providers.
const hasRoutableEmailDomain = async (email) => {
  const normalized = normalizeEmailAddress(email);
  const atIndex = normalized.lastIndexOf("@");
  if (atIndex === -1 || atIndex === normalized.length - 1) {
    return false;
  }

  const domain = normalized.slice(atIndex + 1);
  if (!domain || domain.includes("..") || domain.startsWith(".") || domain.endsWith(".")) {
    return false;
  }

  try {
    const mxRecords = await dns.resolveMx(domain);
    if (Array.isArray(mxRecords) && mxRecords.length > 0) {
      return true;
    }
  } catch {
    // Ignore and fallback to A/AAAA checks.
  }

  const [aResult, aaaaResult] = await Promise.allSettled([dns.resolve4(domain),
```

```
dns.resolve6(domain)]);
  const hasARecords = aResult.status === "fulfilled" && Array.isArray(aResult.value) &&
aResult.value.length > 0;
  const hasAAAARecords =
    aaaaResult.status === "fulfilled" && Array.isArray(aaaaResult.value) &&
aaaaResult.value.length > 0;
```

Appendix C - Schema (server/sql/schema.sql)

```
-- Workflow: Declares MySQL database schema for inventory, logs, images, settings, and billing
tables.
CREATE DATABASE IF NOT EXISTS hardware_store_db CHARACTER SET utf8mb4 COLLATE
utf8mb4_unicode_ci;
USE hardware_store_db;

CREATE TABLE IF NOT EXISTS inventory_records (
  record_id VARCHAR(64) PRIMARY KEY,
  month INT NOT NULL,
  year INT NOT NULL,
  month_label VARCHAR(32) NOT NULL,
  category VARCHAR(120) NOT NULL,
  product_name VARCHAR(255) NOT NULL,
  unit VARCHAR(50) NOT NULL,
  opening_stock_qty DOUBLE NOT NULL,
  stock_added_qty DOUBLE NOT NULL,
  total_available_qty DOUBLE NOT NULL,
  sold_qty DOUBLE NOT NULL,
  remaining_qty DOUBLE NOT NULL,
  unit_price DOUBLE NOT NULL,
  total_sold_value DOUBLE NOT NULL,
  remaining_stock_value DOUBLE NOT NULL,
  reorder_level DOUBLE NOT NULL,
  stock_status VARCHAR(30) NOT NULL,
  supplier_name VARCHAR(255) NOT NULL,
  brand_name VARCHAR(255) NOT NULL,
  marketing_spend_allocated DOUBLE NOT NULL,
  walkins_influenced DOUBLE NOT NULL,
  remarks TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_inventory_product_name (product_name),
  INDEX idx_inventory_month (year, month)
);

CREATE TABLE IF NOT EXISTS stock_logs (
  id VARCHAR(64) PRIMARY KEY,
  log_date VARCHAR(40) NOT NULL,
  month_label VARCHAR(32) NOT NULL,
  product_name VARCHAR(255) NOT NULL,
```

```
  category VARCHAR(120) NOT NULL,
  stock_added_qty DOUBLE NOT NULL,
  sold_qty DOUBLE NOT NULL,
  remaining_qty DOUBLE NOT NULL,
  unit_price DOUBLE NOT NULL,
  notes TEXT,
  supplier_name VARCHAR(255) NOT NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  INDEX idx_stock_logs_product_name (product_name)
);

CREATE TABLE IF NOT EXISTS product_images (
  product_name VARCHAR(255) PRIMARY KEY,
  image_data LONGTEXT NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS app_settings (
  id TINYINT PRIMARY KEY,
  store_name VARCHAR(255) NOT NULL,
  currency VARCHAR(20) NOT NULL,
  categories_json LONGTEXT NOT NULL,
  units_json LONGTEXT NOT NULL,
  price_presets_json LONGTEXT NOT NULL,
  theme VARCHAR(10) NOT NULL,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE IF NOT EXISTS billing_sales (
  id BIGINT PRIMARY KEY AUTO_INCREMENT,
  bill_number VARCHAR(40) NOT NULL,
  sale_date DATE NOT NULL,
  sale_time TIME NOT NULL,
  month_label VARCHAR(32) NOT NULL,
  cu
```

10.2 OPERATIONAL RUNBOOK EXCERPT

Appendix D - Startup Runbook

```
# Project Startup Runbook (Windows)

This document explains exactly how to run the project from scratch after a full laptop
shutdown, so anyone can start it reliably.

## 1) What this project needs to run

- Node.js (LTS recommended)
- npm (installed with Node.js)
- MySQL Server
- VS Code (or another editor)
- Project folder present locally at D:/DA WEB PROJECT

## 2) Why the first step is "Start MySQL Server first"

The backend API connects to MySQL during startup. If MySQL is not running, backend startup can
fail and the app cannot load data.

So "Start MySQL Server first" means:

1. Make sure the MySQL service is running before starting the Node.js app.
2. Confirm the service is in Running state.
3. Only then start the project with npm run dev.

## 3) Detailed MySQL startup steps (Windows)

Use any one method below.

### Method A: Using Windows Services (most common)

1. Press Win + R.
2. Type: services.msc
3. Press Enter.
4. In the Services window, find a service named one of these:
   - MySQL80
   - MySQL57
   - MySQL
5. Check the Status column.
```

6. If Status is not Running:
   - Right-click the MySQL service.
   - Click Start.
7. Optional but recommended:
   - Right-click service -> Properties.
   - Set Startup type to Automatic.
   - Click Apply, then OK.

Expected result: MySQL service status becomes Running.

Method B: Using PowerShell (Admin)

1. Open Start menu.
2. Search PowerShell.
3. Right-click Windows PowerShell -> Run as administrator.
4. List MySQL services:
   - Command: Get-Service *mysql*
5. Start the service (example name MySQL80):
   - Command: Start-Service MySQL80
6. Verify status:
   - Command: Get-Service MySQL80

Expected result: Status shows Running.

If your service name is not MySQL80, replace with the exact name from Get-Service output.

Method C: Using MySQL Workbench check

1. Open MySQL Workbench.
2. Open your local MySQL connection.
3. Run query: SELECT 1;
4. If query runs successfully, MySQL is available.

4) One-time project setup (first day only)

Do this once after downloading/cloning the project.

1. Open VS Code.
2. Open folder: D:/DA WEB PROJECT
3. Confirm environment file:
   - If .env does not exist, create it from .env.example

```
4. Fill database values in .env:
   - DB_HOST
   - DB_PORT
   - DB_USER
   - DB_PASSWORD
   - DB_NAME
5. Install project dependencies:
   - Command: npm install

If PowerShell policy blocks npm, use:
- Command: npm.cmd install

## 5) Daily startup steps (after full shutdown)

Follow these steps in exact order.

1. Start MySQL service first (Section 3).
2. Open VS Code.
3. Open project folder D:/DA WEB PROJECT.
4. Open Terminal in VS Code.
5. Confirm terminal is in project root:
   - Command: pwd
   - Expected path ends with D:/DA WEB PROJECT
6. Check .env exists:
   - Command: Test-Path .env
   - Expected result: True
7. If dependencies changed since last run, reinstall:
   - Command: npm install
8. Start frontend + backend together:
   - Command: npm run dev
   - If execution-policy issue: npm.cmd run dev
9. Wait until startup messages appear:
   - API server running on http://localhost:4000
   - Vite local URL (usually http://localhost:5173, or another available port)
10. Open browser and use the exact Vite Local URL shown in terminal.

## 6) Quick health verification (recommended every run)

After app starts:

1. Open health endpoint in browser:
```

```
   - http://localhost:4000/api/health
2. Expected response: status ok JSON
3. In app UI:
   - Open Dashboard
   - Open Products
   - Open Billing
4. Create a sample bill and verify history row appears.

## 7) If app does not start

### Case A: Backend cannot connect to DB

Possible causes:
- MySQL service is not running
- Wrong DB credentials in .env

Fix:
1. Start MySQL service again.
2. Recheck .env values.
3. Restart app with npm run dev.

### Case B: Port conflict

Possible causes:
- Another process is already using port 4000 or Vite port.

Fix:
1. Stop previous terminals still running project.
2. Restart npm run dev.
3. Use the exact Vite URL shown (it may change to 5174/5175).

### Case C: npm command blocked in PowerShell

Fix:
1. Use npm.cmd instead of npm.
2. Example: npm.cmd run dev

## 8) How to stop project cleanly

1. Go to the terminal where npm run dev is running.
2. Press Ctrl + C.
```

3. Press Y if asked to terminate.
4. Confirm terminal returns to prompt.

9) Optional production check for report/demo

1. Build project:
   - Command: npm run build
2. Run production preview:
   -

10.3 SCREENSHOTS PLACEHOLDER

Insert screenshots for: login, dashboard, products, stock updates, billing history, PDF preview, marketing analytics, inventory analytics, AI forecast lab, and settings. Ensure each figure includes a caption and figure index number.

CHAPTER 11

11. REFERENCES

React Documentation: https://react.dev

Vite Documentation: https://vitejs.dev

TypeScript Documentation: https://www.typescriptlang.org/docs

Express Documentation: https://expressjs.com

MySQL Documentation: https://dev.mysql.com/doc

Node.js Documentation: https://nodejs.org/docs/latest/api

Zustand Documentation: https://docs.pmnd.rs/zustand

Recharts Documentation: https://recharts.org/en-US

Nodemailer Documentation: https://nodemailer.com/about

PDFKit Documentation: https://pdfkit.org

LightGBM Documentation: https://lightgbm.readthedocs.io